Code Jam 2011 - Round 2

Analysis: A.I. War

The computer game A.I. War hides at least a few good algorithmic problems. The author is anything but an expert, but musing about the game brought both this problem and Space Emergency (which originally took place on a graph like this one) to life. The game presents a trickier version of this problem: there are two A.I. homeworlds, multiple other planets that you might want to visit, and you don't know where to find any of them at the start of the game. Fortunately, here we're dealing with a simplified version of the problem and you didn't have to know anything about the game.

Let's first state the problem in graph theory terms. We are given an undirected graph with **P** vertices and **W** edges. We are looking for a sequence of vertices, starting with vertex 0 (our home planet) and ending with a neighbor of 1 (A.I.'s home planet), such that:

- every vertex in the sequence is adjacent to one of the previous vertices
- the sequence is as short as possible
- given the above, the number of distinct neighbors of the vertices in the sequence, but outside the sequence, must be as large as possible

Let **D** be the distance from vertex 0 to vertex 1. It's clear that every such sequence must have at least **D** elements. We also note that we will achieve exactly **D** elements if and only if the sequence forms a shortest path from vertex 0 to vertex 1. Hence we're looking for a shortest path. Unfortunately, there may be many shortest paths and we have to pick the one that optimizes the last requirement.

It simplifies things to include among the "threatened" planets those planets that we do conquer. Given that we already know that we have to conquer exactly **D** planets, we just have to subtract **D** at the end.

Crucial observation

The crucial observation for solving the problem is the following: if a vertex is at distance **d** from 0, it can only be threatened by a vertex at distance **d** - 1, **d** or **d** + 1. This is true because the distances of two adjacent vertices differ by at most 1. Therefore every vertex in the graph is only threatened (or conquered) within at most three consecutive vertices in the path. As we will see, this observation allows us to use dynamic programming to compute best paths of larger and larger lengths.

The algorithm

The first step in our algorithm is to run breadth-first search to compute for every vertex **v** its distance from 0: dist[**v**]. dist[0] = 0 and dist[1] = **D**. Every shortest path will start with vertex 0, then continue with vertices at distance 1, distance 2, ..., distance **D**-1.

For any two adjacent vertices **a**, **b** such that dist[**b**] = dist[**a**] + 1, define **F**(**a**, **b**) = the maximum number of planets threatened or conquered by a shortest path 0→...→**a**→**b**. We will compute this using dynamic programming for increasing distances from 0. The answer to the problem is the maximum value of **F**(**a**, **b**) - **D**, where **a**, **b** are adjacent, dist[**a**] = **D**-2, dist[**b**] = **D**-1, and **b** is adjacent to 1.

F(0, **a**) can be computed directly (there is only one path possible). It remains to compute the values of **F** for a given distance, given the values of **F** at a previous distance. To compute **F**(**b**,

c), dist[**b**]=**d**, dist[**c**]=**d** + 1, try all vertices **a** adjacent to **b** such that dist[**a**] = **d** - 1. In other words, we are looking for paths ending with **a**→**b**→**c**. We have already computed the value of **F**(**a**, **b**) in the previous iteration, the question is: how many new unique threatened vertices does extending the path to **c** add?

This is where our "crucial observation" becomes useful. If a neighbor of **c** was already threatened (or conquered) before, it must have been a neighbor of either **a** or **b**. Therefore we must add the number of neighbors of **c** that are not neighbors of either **a** or **b**. Let's call this value **G**(**a**, **b**, **c**). So we have the recursive formula that we can use for dynamic programming: **F**(**b**, **c**) = $\max_{\mathbf{a}}$ **F**(**a**, **b**) + **G**(**a**, **b**, **c**).

Computing G: four algorithms

We are almost done, but how do we compute the values of **G**, and what is the total run-time of the algorithm? There are O(**W**) values of **F** to compute (at most one for each edge), and for each one we look at O(**P**) other values of **F** (one for each **a**). So if we already knew all the values of **G**, it would take O(**PW**) time to compute all the values of **F** and solve the problem. Computing the values of **G** turns out to be the most time-consuming part though.

We may have to compute **G**(**a**, **b**, **c**) for every sub-path **a**→**b**→**c** of increasing dist. How do we compute these values? We have thought of at least 4 ways. In the actual contest it didn't really matter which you chose, all would easily run in time, but we will mention them just for fun.

- **Approach 1.** We need to compute the number of vertices **v** such that **v** is a neighbor of **c**, but not of **a** or **b**. The simplest approach here is to just check this condition for every **v**. This computation takes O(**P**) time. How many values do we need to compute? At most O($\mathbf{P}^3$), which gives a total run-time of O($\mathbf{P}^4$) for the whole algorithm. We can give a better estimate. Since **a** and **b** are neighbors, there are at most **W** such pairs. This gives the total run-time of O($\mathbf{P}^2$ **W**).
- **Approach 2.** Modify approach 1 slightly. Instead of checking every vertex **v**, we only need to check every neighbor of **c**. This way there are O(**W**) pairs **a**, **b** and O(**W**) pairs **c**, **v**, which gives the total run-time of O($\mathbf{W}^2$).
- **Approach 3.** Modify approach 1 in another way. Precompute the set of neighbors of each vertex as a bitmask neighbors[**v**]. Now we are simply interested in the number of bits set in (neighbors[**c**] **and not** (neighbors[**a**] **or** neighbors[**b**]). This is a speedy computation due to bit-level parallelism. If our computer has machine words of size **w** (this is typically 32 or 64), we cut our work by a factor of **w**. This assumes that we can count the number of bits set in a word in a single step, which modern processors support in a single machine instruction. The final runtime: O($\mathbf{P}^2$ **W** / **w**).
- **Approach 4.** Finally, we come to a theoretically interesting, but rather complex to implement aproach. Define two matrices, **A** of size **W** x **P**, and **B** of size **P** x **P**, as follows. $\mathbf{A}_{\mathbf{ij}}$ = 1 if vertex number **j** does not neighbor any of the two endpoints of the edge number **i**, 0 otherwise. $\mathbf{B}_{\mathbf{ij}}$ = 1 if vertices number **i**, **j** are adjacent (or equal), 0 otherwise. Now compute the matrix product **C** = **A** * **B**. If we apply the definition of matrix product and do the math, we will see that **C** is a **W** x **P** matrix and that the entries are exactly the values of **G**. Specifically, $\mathbf{C}_{\mathbf{ij}}$ = **G**(**a**, **b**, **j**) if the **i**-th edge is **a**↔**b**.
  If we evaluate the matrix product in the natural way, we get the same complexity as in approach 1: O($\mathbf{P}^2$ **W**). We have basically expressed approach 1 in matrix notation.
  The trick is that there are faster matrix multiplication algorithms. The fastest currently known is the Coppersmith-Winograd algorithm. To make a long story short, it gives us a theoretical asymptotic run-time of O($\mathbf{P}^{1.376}$ **W**).

We do not recommend the last approach in the contest. Not only is this unnecessarily complicated, it also wouldn't actually run any faster for the graph sizes we are considering. The

asymptotic advantage would only materialize for impractically enormous, dense graphs.

Code Jam 2011 - Round 2

Analysis: Airport Walkways

Some people will tell you that programming contest problems, while interesting to think about, have no practical use in the real world. After solving this problem, you can laugh at these people as you catch your flight and they are left waiting in line at the airport with all the other chumps.

An important thing to notice about this problem is that the location of the walkways does not matter, since you can instantly transition between different speeds. This means that two walkways with speed **v** of length L_1 and L_2 can be combined into a single walkway of length $L_1 + L_2$ with speed **v**. By combining this with the observation that any section of corridor with no walkway is equivalent to a walkway with **v = 0**, you reduce the problem to having 101 different speed walkways of variable length (possibly 0) and deciding for each whether to run or walk (or do some of each).

The small dataset can be solved by brute force. Let **W** be the set of all positive length walkways (each with a unique speed). Since there are only **|W| ≤ 21** different speed walkways to consider, you can simply try choosing each subset **S ⊆ W** of them to run (and just walk the rest, **W - S**). This solution is slightly complicated by the fact that once you choose **S**, you still have to decide which one to only run partially, in case you donâ€™t have time to run them all fully. However, you can again just brute force this decision by iterating over each walkway **x ∈ S** and only running on walkway **x** with whatever time is left after completely running all walkways in **S - {x}**. You simply take the minimum over all choices, discarding any choice which requires more than **t** seconds of running time. This can easily be implemented in $O(N^2 * 2^N)$ and will run in time for **N** at most 20.
Bonus: Implement this algorithm in $O(N * 2^N)$ time.

However, this approach will time out for the large data set, where **N** is at most 1000 and you can have walkways of all 101 different speeds. For this, we need a better way to decide when to run and when to walk. We just need to decide if itâ€™s better to run on slower or faster walkways. It turns out we can solve this with a simple greedy algorithm, and we can prove it is optimal by using a simple exchange argument.

Letâ€™s say we have two arbitrary walkways of speed w_1 and w_2, with $w_1 < w_2$. Now letâ€™s say that in some algorithm weâ€™ve decided to run for r_1 and r_2 seconds on each of the walkways, respectively. If s_1 and s_2 are the amount of time we spent walking on each of the two walkways, then our total time spent would be $T = r_1 + s_1 + r_2 + s_2$ seconds. What if instead we decided to run for $r_1 + \varepsilon$ seconds on the first walkway and $r_2 - \varepsilon$ seconds on the second walkway, with $\varepsilon > 0$? Then our total time would be $Tâ€™ = (r_1 + \varepsilon) + s_1â€™ + (r_2 - \varepsilon) + s_2â€™$ seconds. Solving for $T - Tâ€™$, you will get $\varepsilon * (w_2 - w_1) * (R - S) / ((w_1 + S) * (w_2 + S)) > 0$, which says that $T > Tâ€™$, and so the change will always be beneficial. Simply put, itâ€™s always better to run on slower walkways as much as possible. *Note that some of the details of these equations have been excluded from this analysis and are left as an exercise to the reader.*

Here is some simple Java code which solves the problem:

```
double res=0;
for (int i=0; i<=100; ++i) {
  double runTime=Math.min(t,W[i]/(i+R));
  double walkDist=W[i]-runTime*(i+R);
  double walkTime=walkDist/(i+S);
  res+=runTime+walkTime;
```

```
  t-=runTime;
}
System.out.printf("Case #%d: %.12f%n",TC,res);
```

Bonus: Solve the problem if the limits changed to $1 \leq w_i \leq 10^9$.

Code Jam 2011 - Round 2

Analysis: Expensive Dinner

This problem might look pretty intimidating at first:

- There are **N**! possible orders in which your friends could arrive.
- Not only is O(**N**!) too slow for this problem, even O(**N**) is too slow!
- The actual price your friends are paying will quickly exceed the bounds of a 64-bit integer.

When O(**N**) is too slow, it means you need to forget about coding for a while and think. You will need some insight to even get started.

Let's begin by fixing an ordering (x_1, x_2, ..., $x_{\mathbf{N}}$) of your friends. Also let y_i be the total price that your group is paying after the first i friends enter, the waiter has been called if necessary, and everyone has become happy.

Observation 1: $y_i = \text{LCM}(x_1, x_2, ..., x_i)$. This does not depend on the order your friends buy things after the waiter has been called.

Explanation: $\text{LCM}(x_1, x_2, ..., x_i)$ is, by definition, the smallest multiple of x_1, x_2, ..., x_i. Since y_i must be a multiple of all these numbers for your friends to be happy, it is certainly true that $y_i \geq \text{LCM}(x_1, x_2, ..., x_i)$. Conversely, a friend x_j would never buy something that skips over a multiple of x_j. In particular, none of the friends here would buy something that would skip over $\text{LCM}(x_1, x_2, ..., x_i)$. Since $y_{i-1} \leq \text{LCM}(x_1, x_2, ..., x_i)$, you will eventually reach this price and then stop.

Let **M** be the number of times the waiter is called over. Then, **M** is equal to the number of values i in {1, 2, ..., **N**} such that $y_i \mathrel{!}= y_{i-1}$. (We define $y_0 = 0$, because the waiter is always called over by the first friend who enters.) So the question becomes: how do we make **M** as big as possible according to this definition, and how do we make it as small as possible?

Least common multiples are closely related to prime numbers and factorizations, so let's define p_1, p_2, ..., $p_{\mathbf{P}}$ to be the primes less than or equal to **N**. Also let e_i be the largest integer such that p_i^{ei} is less than or equal to **N**.

Observation 2: Let **P'** be 1 + sum e_i (i.e., the number of prime powers less than or equal to **N**, including 1). Then the maximum value of **M** is exactly equal to **P'**.

Explanation: Suppose your friends arrive in the order 1, 2, ..., **N**. Then each friend with a prime power index will cause the price to increase, and so the maximum value of **M** is at least **P'**.

On the other hand, the price is always a least-common multiple by the first observation, and it always increases to a multiple of itself. In particular, the sum of the exponents in its prime factorization must always go up every time the waiter is called. After the first friend arrives, this sum is at least 0. At the very end, it is equal to **P'** - 1. Therefore, the waiter can be called at most **P'** - 1 times after the first person arrives. Combining that with the initial increase proves that **M** ≤ **P'**.

Observation 3: If **N** > 1, then the minimum value of **M** is equal to **P**.

Explanation: Suppose the first friends to arrive are numbered p_1^{e1}, p_2^{e2}, ..., $p_{\mathbf{P}}^{e\mathbf{P}}$. Then the price is already equal to LCM(p_1^{e1}, p_2^{e2}, ..., $p_{\mathbf{P}}^{e\mathbf{P}}$) = LCM(1, 2, ..., **N**) after these friends have arrived. This means no subsequent friend will change the total price, and hence **M** = **P** in this case.

On the other hand, notice there is no number less than **N** that is divisible by both $\mathbf{p}_i^{ei}$ and $\mathbf{p}_j^{ej}$. This is because $\mathbf{p}_i^{ei}$ and $\mathbf{p}_i^{ej}$ are relatively prime and larger than sqrt(**N**). (If one of them was at most sqrt(**N**), then its exponent could be increased, which is a contradiction.) Therefore, no single friend can make the total price divisible by two different entries out of {p_1^{e1}, p_2^{e2}, ..., $p_{\mathbf{P}}^{e\mathbf{P}}$}. And so the waiter must be called at least **P** times, as claimed.

To solve the small input, you can just calculate **P** and **P'** directly and go from there. The large input requires one last clever trick. The number of primes less than 10^{12} is quite large, and you probably cannot afford to just count them all. However, calculating **P** - **P'** is actually easier than this!

If you go back to the original definitions, you can see **P** - **P'** is precisely the number of integers less than or equal to **N** that can be written in the form p^e for e != 1. When e = 0, you just get 1. When e > 1, then p ≤ 10^6, and you can easily enumerate all primes of this size! One good way is to use the Sieve of Eratosthenes. As always, you can look at solutions from other contestants to see a full implementation.

Bonus Comment: The Sieve of Eratosthenes runs in O(**M** * log log **M**) time, so the simplest implementation of the method described here runs in O(sqrt(**N**) * **T** * log log **N**) time altogether. However, it's worth pointing out that you can do even better. If you pre-compute and sort all prime powers less than 10^{12} with exponent other than one, you can then use a binary search to very efficiently count how many are less than **N** for each test case. This is not necessary to solve the problem, but it lets you speed things up even more to a blazing O(sqrt(**N**) * log log **N** + **T** * log **N**). Proving this running time is a little tricky though!

Code Jam 2011 - Round 2

Analysis: Spinning Blade

Let us begin by thinking how can we determine the center of mass of a given blade. The simplest formula for the center of mass (obtained by transforming the formula from the problem statement) is: **sum(Mass$_i$ * P$_i$) / sum(Mass$_i$)**, where **P$_i$** is the position of cell **i** relative to a static location, such as the upper-left corner of the sheet of metal, **Mass$_i$** is the mass of cell **i**, and **i** is iterated over all cells in the blade.

The first thing to note about this problem is that the X and Y coordinates of the center of mass can be calculated independently, which will simplify the calculations significantly. The X and Y coordinates of the center of the blade are also easy to calculate (the X coordinate is the average of the smallest and largest X coordinate of any cell in the blade).

The thing we are interested in is whether the center of the blade and the center of mass of the blade coincide. To avoid floating point calculations (which would induce the need to think about possible precision problems) we can multiply the equality **sum(Mass$_i$ * X$_i$) / sum(Mass$_i$) = (minX + maxX) / 2** by the denominators of both sides. Thus, for each blade we need to check the equality **2 * sum(Mass$_i$ * X$_i$) = (minX + maxX) * sum(Mass$_i$)**.

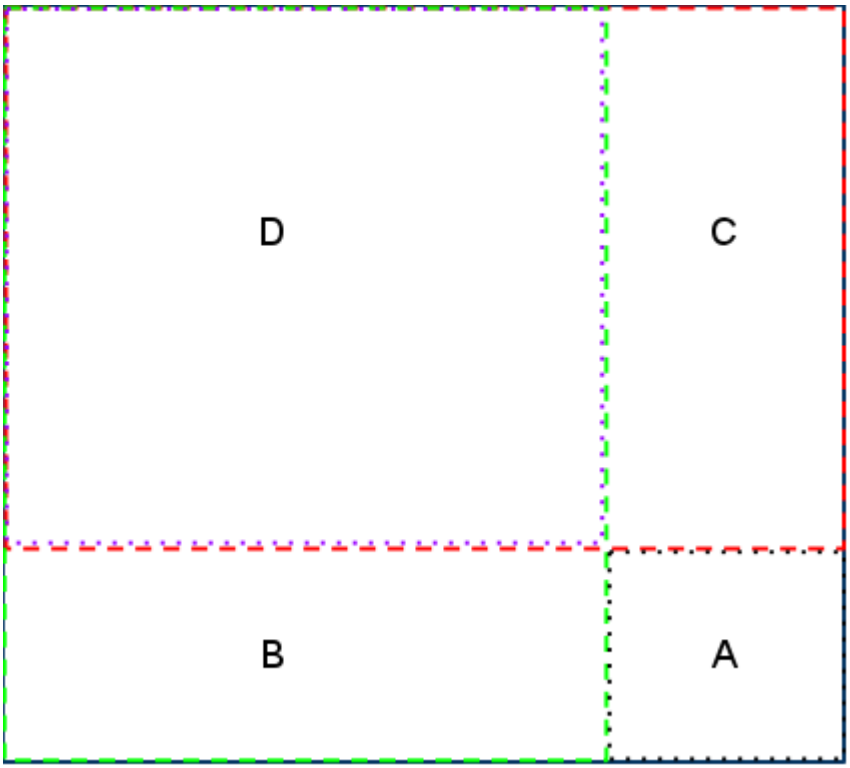
So, we simply need to test this equality for all possible blades. This can be done by iterating over all possible X and Y coordinates of the upper-left corner of the blade, and then over all possible sizes of the blade. Calculating either side of the equality above can be performed by iterating over all cells in the blade (remember to omit the corners!). As there are O(RC) possible upper-left corners, O(min(R,C)) possible sizes and O(min(R,C)^2) cells in a blade, the whole algorithm has a time complexity of O(N^5) (where **N** denotes the common upper bound for **R** and **C**), which works for the small input, but we cannot expect to make it work for the large.

Before we attack the large case, let us spend a moment to look at potential overflow problems — we already saw that this can be a serious issue in this competition! The left-hand size of the inequality can be estimated by **2 * N^2 * N * maxW** — two times the number of cells times the largest possible value of **X$_i$** times the largest possible weight of a cell. In the small test cases, this value will easily fit into a 32-bit integer, while in the large case we should use 64-bit integers to be safe. We also considered giving a 10^{18} bound on **D**, several approaches of dealing with overflow (that can also handle this obscenely large limit) are given at the end of this editorial, you might also want to think about this problem yourself.

Back to the large case. Notice that in the previous approach we have seen there are O(N^3) blades to consider, so one approach to reducing the run time is to attempt to make the center of mass calculation for every blade constant. This requires a bit of precalculation.

To precalculate the center of mass (or rather, the **sum(Mass$_i$ * X$_i$)** and **sum(Mass$_i$)** quantities) for any given blade, we will first calculate the center of mass and total mass for all rectangles with the two corners at (0,0) and (x,y). We will start with the rectangle with corners at (0,0) and (1,1). This is just the first cell of the grid, so we already know its center of mass and total mass. We will store this answer and move on to the next rectangle we want to calculate, which will have corners at (0,0) and (1,2). Since we already know the center of mass and total mass for the rectangle with corners (0,0) and (1,1) we can use this rectangle, and the rectangle with corners at (0,1) and (1,2) to calculate the center of mass and total mass for the rectangle with corners (0,0) and (1,2). As long as we iterate over the Y axis 1 by 1 we can calculate the total mass and center of mass of all possible rectangles in constant time.

Now we have to handle the case where we have to iterate the X axis of the corner, so we need to know the center of mass and total mass of the rectangle with corners at (0,0) and (2,1). This is the same as the first case for the Y axis, so we just calculate the center of mass and total mass using the rectangle with corners at (0,0) and (1,1) as well as the rectangle with corners at (1,0) and (2,1). In the next step we run into a problem. We have a grid that looks something like the image below.



If it is not clear in the image, the rectangles marked B and C overlap with the rectangle D.

We know the appropriate sums for the rectangles marked A, B, C, and D, but we need to know the sums for the entire rectangle, which we will call R. This can be done in constant time using:

R = A + B + C - D

We subtract D because it is added twice when we add both B and C.

This can be used to calculate center of mass and total mass both for all rectangles with corners (0,0) and (2,2) on to (X,Y). The total run time for this is $O(N^2)$ since we only have to look at each cell once. Now, how can this be used to calculate the sums for a square with corners at (x_1,y_1) and (x_2,y_2) (or any other square we might be interested in)? This is very similar to the way we calculated the center of mass and total mass during the precalculations. Assume we label a few of the rectangles we have already precalculated as:

A = Square we are looking for, with corners at (x_1,y_1) and (x_2,y_2)

B = Rectangle with corners at (0,0) and (x_1,y_2)

C = Rectangle with corners at (0,0) and (x_2,y_1)

R = Rectangle with corners at (0,0) and (x_2,y_2)

D = Rectangle with corners at (0,0) and (x_1,y_1)

The same picture, reasoning and equation as before gives us **R = A + B + C - D**, which transforms to **A = R + D - B - C**. Thus, having all the precalculations done, we can compute all the needed quantities for any square (and thus, by subtracting the values in the corners, for any blade) in constant time!

With a constant center of mass calculation, this brings the total run time to $O(N^2 + N^3)$. This will easily work with N <= 500.

Side Note

Another approach to this problem is to try all possible upper-left corners for the blade, and then slowly expand the size of the blade by 1 unit to the bottom-right at a time until it cannot be expanded any further, calculating the required sums using O(N) calculations at each step to increment the previous values. That brings the total run time to $O(N^4)$. While this may seem incredibly large when N can be up to 500, in practice this is really a lot less calculations than 500^4 (as for most corners we cannot expand up to size N), while the input file size limit guarantees there are at most two max-cases in the input. Thus, this approach will run in time on most computers and in most languages.

Dealing with overflow

Let us go back to the question what would we do if the limit on **D** was larger.

1. We can try to use BigIntegers of some sort. This solution costs us in terms of efficiency, although if we go for the $O(N^3)$ solution we should still be able to make it.
2. We can realize that the value of **D** is irrelevant. On an intuitive level - adding a constant mass **D** to each cell is the same as putting a new sheet with each cell of mass **D** on top of our sheet. The center of mass is going to be somewhere in between the centers of mass for the two sheets — and so it is going to be in the middle if and only if our original sheet had the center of mass in the middle. On a formal level, if we add **D** to each $\mathbf{Mass_i}$, both sides of the equation increase by **2 * AvgX * NumberOfCells**, where **AvgX** is the average X coordinate of a cell in the blade. Thus, we can set **D** to, say, 1 and use 32-bit integers
3. We can also set **D** to zero. This is somewhat scary, as it would result in a division by zero in the original equation for the center of mass, we encourage you to consider why this works.
4. Finally, we may ignore the overflow problem totally, if only our compiler guarantees that overflow results in modulo arithmetics. As **D** cancels out on both sides of the equation anyway, it also cancels out modulo MAXINT, so if only the rest of the equation will not overflow, we will be fine. This requires some knowledge of your tools, though — for instance modular arithmetic for integers in C++ is guaranteed for unsigned integers, but not for signed integers, and the compiler can make optimizations that assume overflow does not happen for signed integers (and thus break code depending on the modular arithmetic).